

IAM Hardening Summary - Northstar Assist

Tarie Nosworthy

18 September 2026

Contents

Northstar Assist — Least-Privilege Access Controls	1
0. Erratum — the hardening widened four logging statements	1
1. Method	2
2. Roles reviewed	3
A finding about where the permissions actually live	4
3. Changes applied	4
3.1 Harness execution role	4
3.2 Gateway service role — 2 bare wildcards to 0	5
3.3 Knowledge base service role	6
4. Surviving wildcards — individual justification	6
4a. Bare Resource: "*" — eight statements	7
4b. Partial wildcards — seven statements, omitted from the earlier draft	7
5. Verification — and why it was built in	9
6. What an attacker gains — before versus after	9
7. Remaining recommendations	10
Added after post-submission review (19 September 2026)	10
Evidence	10

Northstar Assist — Least-Privilege Access Controls

Assessment date: 18 September 2026 **Author:** Tarie Nosworthy **Environment:** AWS account 118924230273, us-east-1

Every policy document quoted here was **read from the live account**, narrowed programmatically, applied, and verified. Nothing was retyped. The complete before/after documents and unified diffs are in `submission/iam/`, the machine-readable change log in `evidence/iam/iam_changes.json`, and the generator's own output in `evidence/iam/generated-change-log.md`.

0. Erratum — the hardening widened four logging statements

Found in post-submission review, 19 September 2026. This corrects a claim made elsewhere in this document and in the threat model.

Section 3.1 below describes the CloudWatch Logs changes as narrowing. **For three of the four statements that is wrong — they were widened.** Dified from the applied policy:

Statement	Before	After (applied)	Effect
CloudWatchLogsGroup	...:log-group:/aws/bedrock-agentcore/runtimes/*	...:log-group:/aws/bedrock-agentcore/*	widened
CloudWatchLogsStream	.../runtimes/*:log-stream:*	.../aws/bedrock-agentcore/*	widened
CloudWatchLogsPutResourcePolicy	.../runtimes/harness_NorthstarAssist-*	.../aws/bedrock-agentcore/*	widened
CloudWatchLogsDescribeGroups	...:log-group:*	the narrowed pair	narrowed (correct)

All four additionally gained `arn:aws:logs:us-east-1:118924230273:log-group:/northstar-assist/model-invocations:*`.

Why this matters more than the ARN strings suggest. That last grant gives the agent’s own execution role **write access to the log group recording its own model invocations** — the audit trail of its behaviour. Threat **R-02** (“Log tampering by the agent’s own role”) is marked *mitigated* in the threat model on the strength of this work. It was not mitigated; it was made worse. R-02 is re-rated **REGRESSED** accordingly.

Root cause. `resolve_targets()` returned a single logs ARN list, and `narrow_policy()` applied it to every statement whose actions mapped to the logs group, discarding each statement’s original, tighter scope. The deeper fault is that **the narrowing pass never verified it had narrowed** — it computed a replacement resource set and wrote it without comparing it to the original.

Fix applied to the code, not to the evidence. `scripts/20_harden_iam.py` now carries an `is_narrower()` guard: every proposed ARN must be covered by at least one original ARN, or the statement is left exactly as the console wrote it and flagged KEPT-WOULD-WIDEN. Leaving a permission too broad is recoverable; silently broadening one is not. The harness logs target list no longer contains the model-invocation group at all, because Bedrock delivers those records through `NorthstarAssistBedrockLoggingRole` and the harness has no reason to write there.

Re-running the corrected narrowing against the captured before-policy now produces:

```

NARROWED      CloudWatchLogsDescribeGroups  log-group:* -> runtimes/*
NARROWED      CloudWatchLogsGroup            runtimes/* (kept, +log-stream scope)
KEPT-WOULD-WIDEN CloudWatchLogsStream        original preserved
KEPT-WOULD-WIDEN CloudWatchLogsPutResourcePolicy original preserved

```

These corrected statements were NOT re-applied. The AWS environment was torn down before the review, so `iam/after/` remains an accurate record of what was actually applied on 18 September, including the regression. It has deliberately not been hand-edited — a before/after evidence set that has been retouched is worth nothing. Re-applying and re-verifying is the first item in §7.

What this says about the rest of the document. The other narrowing claims were re-checked against the applied policy and hold: the model-invocation narrowing, the guardrail scoping, the seven removed statements, and the `aws-marketplace:Unsubscribe` removal are all as described. The failure was specific to the logs group, and it was caught by external review rather than by my own verification — which checked that *the agent still worked*, not that *every permission had actually shrunk*. A verification step that only asks “is it still running?” cannot catch a widening.

1. Method

The policies were narrowed, not replaced.

The obvious approach — write ideal policies from scratch and overwrite the console’s — has two problems. It requires inventing IAM action names, some of which do not exist. And it produces no honest before/after comparison, because the “before” becomes a description rather than a document.

Instead scripts/20_harden_iam.py reads what the console actually attached and applies these transformations:

Transformation	Rule
NARROWED	A statement granting an action on a broad resource is rewritten to the specific ARNs this system uses, resolved from live state
SPLIT	A statement whose actions span several resource groups is divided so each carries its own scope
REMOVED	A statement whose actions serve only a feature that was never configured is deleted entirely
ACTION-REMOVED	An individually indefensible action is stripped from an otherwise legitimate statement
UNCHANGED-WILDCARD	Flagged for explicit justification — never silently accepted

Two design details that mattered in practice:

Feature detection is by predicate, not by an action list. The console grants GetBrowserSession, UpdateBrowserStream, s3files:ClientMount and similar. A hardcoded list drifts out of date the moment AWS adds an action, and the first attempt here matched none of the real names. The matchers key on each feature’s vocabulary instead.

The model ARNs come from the inference profile itself. A cross-Region profile requires both the profile ARN *and* the regional foundation-model ARNs it can route to — granting only the profile produces AccessDenied at invoke time. Rather than assume which regions, the script reads them from the live profile. This one routes to **three** regions; a naive us-* guess would have granted four, and the fourth would have been an unused permission.

2. Roles reviewed

Role	Purpose	Statements removed	re-narrowed	Wildcards before	Wildcards after
arn:aws:bedrock:us-east-1:123456789012:agent/core/harness/default-service-role-fpraf	Ultimate service role — invoke model, apply guardrail, invoke gateway, write logs	7	5	6	6 (all justified, §4)
arn:aws:bedrock:us-east-1:123456789012:agent/core/gateway/default-service-role-1789761291328	GetRole — retrieve from the knowledge base	2	3	2	0
arn:aws:bedrock:us-east-1:123456789012:agent/core/knowledge-base/pepa3	KBBase:pepa3 — read S3, invoke the embedding model	1 action	1	2	2 (both justified, §4)

Totals: 9 statements removed, 9 narrowed or split, 1 action stripped, 28 individual permissions withdrawn, 8 bare wildcards and 7 partial wildcards surviving, each justified in §4.

⚠ The “wildcards after” column counts only bare Resource: `"*"`. It does not count partial wildcards (a `*` inside an otherwise-scoped ARN), which is why the gateway row reads 0 while three partial wildcards remain — see §4b. And three of the harness row’s statements were **widened**, not narrowed; see §0.

A finding about where the permissions actually live

The project brief implies the over-permission is in inline policies. **It is not.** The console-generated AgentCore roles put almost everything in **customer-managed attached policies**:

Role	Inline	Attached	Where the wildcards were
Harness	1	2	All 6 in the attached <code>...HarnessExecutionPolicy_le77v</code>
Gateway	0	2	Both in attached policies
Knowledge Base	0	3	Both in attached policies

This matters operationally: narrowing them requires `CreatePolicyVersion(SetAsDefault=True)`, not `PutRolePolicy`. A hardening script that only handles inline policies runs successfully, reports changes, and **leaves every real over-permission in place**. The first version of this script did exactly that.

3. Changes applied

3.1 Harness execution role

NARROWED — `BedrockModelInvocation` (*the headline change*)

Before:

```
"Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
"Resource": [
  "arn:aws:bedrock:*:foundation-model/*",
  "arn:aws:bedrock:us-east-1:118924230273:*"
]
```

After:

```
"Resource": [
  "arn:aws:bedrock:us-east-1:118924230273:inference-profile/us.amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-east-1:foundation-model/amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-east-2:foundation-model/amazon.nova-2-lite-v1:0",
  "arn:aws:bedrock:us-west-2:foundation-model/amazon.nova-2-lite-v1:0"
]
```

Rationale. The second ARN in the original — `arn:aws:bedrock:us-east-1:118924230273:*` — is **every Bedrock resource in the account**: every model, guardrail, knowledge base and inference profile, including resources that do not exist yet. The harness invokes exactly one inference profile. Granting `foundation-model/*` would additionally let a compromised role invoke every model in the account, including far more capable and more expensive ones.

NARROWED — `ApplyNorthstarGuardrail`

Scoped to one guardrail **and its cross-Region profile across three regions**:

```
arn:aws:bedrock:us-east-1:118924230273:guardrail/6yxbopugn44t
arn:aws:bedrock:us-east-1:118924230273:guardrail-profile/us.guardrail.v1:0
arn:aws:bedrock:us-east-2:118924230273:guardrail-profile/us.guardrail.v1:0
arn:aws:bedrock:us-west-2:118924230273:guardrail-profile/us.guardrail.v1:0
```

Rationale, and a finding. `ApplyGuardrail` on `*` would let the role apply **any** guardrail in the account — including a permissive one, which defeats the control it is supposed to enforce.

The four ARNs rather than one is not padding. A STANDARD-tier guardrail runs through cross-Region inference, so authorisation covers the guardrail *and the profile* — and **the profile ARN is regional, following request routing**. Granting only us-east-1 produced an intermittent failure when a call routed to us-east-2:

```
not authorized to perform: bedrock:ApplyGuardrail on resource:
arn:aws:bedrock:us-east-2:118924230273:guardrail-profile/us.guardrail.v1:0
```

The regions are read from the model's inference profile — the guardrail is applied wherever inference runs, so those are exactly the regions needed. Still enumerated; no wildcard. The project brief's sample policy grants only the guardrail ARN and would fail on any STANDARD-tier guardrail.

CloudWatch Logs statements — ⚠️ SEE ERRATUM §0

Only one of the four logging statements was genuinely narrowed. The other three were widened, and all four gained write access to the model-invocation log group. Full detail, root cause and the code fix are in §0; this section is left in place rather than rewritten so the error and its correction sit side by side.

Statement	Intended	Actually applied
CloudWatchLogsDescribeGroups	narrow from log-group:*	✅ narrowed as intended
CloudWatchLogsGroup	keep runtimes/*	❌ widened to bedrock-agentcore/*
CloudWatchLogsStream	keep runtimes/*:log-stream:*	❌ widened to bedrock-agentcore/*
CloudWatchLogsPutResourcePolicy	keep harness_NorthstarAssist-*	❌ widened to bedrock-agentcore/*

The valid part of the rationale. logs:DescribeLogGroups on log-group:* let the agent's own role enumerate every log group in the account, and narrowing that was correct. The irony is not lost: the statement immediately after it in the same policy reasons about anti-forensics while the change being made handed the agent write access to its own audit log.

REMOVED — 7 statements, 28 permissions, all for features that are off

Statement	Actions	Why removed
AgentCoreBrowserDefault	7	Browser tool is off on the harness
AgentCoreCodeInterpreterDefault	5	Code Interpreter is off
AgentCoreMemory	5	Memory is disabled — no events to read or write
EFSClientAccess	2	No EFS mount configured
EFSDescribe	2	As above
S3FilesClientAccess	3	No S3 file-system mount configured
S3FilesDescribe	2	As above

Rationale. A permission with no caller is pure attack surface. These are exactly what a compromised role reaches for: StartBrowserSession gives outbound web access under the agent's identity; InvokeCodeInterpreter gives arbitrary code execution; the file-system mounts give persistent read/write storage. None is needed to answer a policy question, and each was granted by default.

3.2 Gateway service role — 2 bare wildcards to 0

REMOVED — AllowBedrockAgenticRetrieveFromKnowledgeBase, AllowBedrockRerankForKnowledgeBase

Both granted bedrock:AgenticRetrieveStream / bedrock:Rerank on "*".

Rationale. The gateway target is configured for **Standard retrieval**, not Agentic retrieval. Neither action has a caller in this deployment, and both were granted on every resource.

Correction on the claim. An earlier draft said this took the role to “zero wildcards.” That was wrong, and it was wrong because the counter behind it only matched a bare Resource: `“*”`. Removing those two statements took the role to **zero unscoped wildcards**; three *partial* wildcards remain and are justified in §4:

- `bedrock-agentcore:GetConfigurationBundleVersion` on `configuration-bundle/*`
- `bedrock:GetInferenceProfile` on `inference-profile/*`
- `bedrock-agentcore:GetGateway` on `gateway/northstar-assist-gateway-ljkdshz4iy/*`

The third is scoped to this gateway and is genuinely tight. The first two are not, and are recorded as such.

NARROWED — AllowBedrockInvokeModelForKnowledgeBase

From `foundation-model/*` plus `inference-profile/*`, `provisioned-model/*` and `custom-model/*` — to the Titan embedding ARN only.

Rationale. The gateway touches models solely for retrieval-side embedding. Granting it the generation model would let a compromised retrieval path run inference the design never intended. This is why the script resolves targets **per role** rather than globally: “the model” means different things to the harness and to the gateway.

NARROWED — AllowBedrockApplyGuardrailForKnowledgeBase

From `guardrail/*` to the specific guardrail.

3.3 Knowledge base service role

Already well scoped by the console — `bedrock:InvokeModel` on the exact Titan ARN, `s3:ListBucket` and `s3:GetObject` on the exact bucket. One change needed.

ACTION-REMOVED — aws-marketplace:Unsubscribe

Before: `["aws-marketplace:Subscribe", "ViewSubscriptions", "Unsubscribe"]` on `“*”` **After:** `["aws-marketplace:Subscribe", "ViewSubscriptions"]` on `“*”`

Rationale. Unsubscribe on `*` lets this role **cancel foundation-model subscriptions for the entire account** — an availability attack against every AI workload in the account, not just this one. The knowledge base role needs to read and create subscriptions in order to use a model; it never needs to revoke one.

On why Subscribe and ViewSubscriptions were kept, rather than the whole statement removed. This is the weakest justification in the document and is flagged as such. The knowledge base uses **Titan Embeddings V2**, an AWS first-party model that needs no Marketplace agreement, so on the evidence available this statement has no caller at all and the stronger move is to remove it entirely — as was done for Browser, Code Interpreter and the rest. It was kept because the Marketplace interaction sits inside the managed Knowledge Base service rather than in code I could inspect, and removing a permission a managed service might take on first use risked breaking ingestion with the environment already torn down and no way to re-verify. **Recommended: remove the statement and re-run a sync to confirm.** Listed in §7.

A causal claim in an earlier draft was overstated and is withdrawn. That draft said this statement “explains” why Anthropic models failed — the harness role lacking marketplace permissions while the KB role held them. The invoke error does name `aws-marketplace:ViewSubscriptions`, `aws-marketplace:Subscribe` against the harness role, so the role genuinely lacks them. But `evidence/model_access.json` shows `agreementAvailability.status = NOT_AVAILABLE` for all three Anthropic models, which is an **account-level** entitlement state, not a role-policy one. Whether granting the harness role those actions would have let it auto-subscribe was never tested, and if no agreement is available to create then no role policy would have helped. The accurate statement is: **the role lacks the permission, and the account lacks the agreement; the account-level state is the more likely binding constraint, and the role-policy theory is untested.**

4. Surviving wildcards — individual justification

The rubric requires that no wildcard remain unless explicitly justified. There are two distinct kinds, and an earlier draft only accounted for the first.

4a. Bare Resource: "*" — eight statements

Every one is on an action for which AWS does not support resource-level permissions — the wildcard is not a shortcut, it is the only valid form.

Statement	Actions	Justification
CloudWatchMetricsPublish (harness)	cloudwatch:PutMetricData	No resource-level permissions supported. Constrainable only by a cloudwatch:namespace condition key; recommended as a follow-up.
CloudWatchWritePermissionStatement (KB)	cloudwatch:PutMetricData	As above.
XRayTracingAccess	xray:PutTraceSegments, PutTelemetryRecords, GetSamplingRules, GetSamplingTargets	X-Ray supports no resource-level permissions for these. Write-only telemetry; no read of other services' traces.
EcrPublicTokenAccess	ecr-public:GetAuthorizationToken	Token-issuing call with no resource to name. Required to pull the harness runtime image.
EcrManagedImageToken	ecr:GetAuthorizationToken	As above. The paired EcrManagedImagePull is scoped, to repository/harness-.*.
StsForEcrPublicPull	sts:GetServiceBearerToken	Bearer-token issuance for the ECR pull; no resource.
BedrockMantleCallWithBearerToken	bedrock-mantle:CallWithBearerToken	Bearer-token call with no resource to name.
MarketplaceOperationsFromBedrockFor3pModels	aws-marketplace:Subscribe, ViewSubscriptions	Account-scoped by nature. Unsubscribe was removed — the dangerous member of the set.

Four of these (ecr-public, ecr, sts:GetServiceBearerToken, bedrock-mantle) are the bootstrap that starts the harness runtime. Removing them would stop the agent running at all — which the verification in §5 would have caught.

4b. Partial wildcards — seven statements, omitted from the earlier draft

These carry a * inside an otherwise-scoped ARN. They are **not** covered by the “no resource-level permissions” argument above, and leaving them out of the justification table was an omission, not a judgement.

Role	Statement	Resource	Assessment
Harness	AgentCoreWorkloadIdentity	...workload-identity- directory/default/ workload-identity/harness_NorthstarAssist- *	Tight. Scoped to this harness's own identity; the * covers only the generated suffix.
Harness	EcrManagedImagePull	arn:aws:ecr:us- east-1:*:repository/ harness-*	Acceptable, imperfect. Repository prefix is scoped; the * is the <i>account</i> field, because AWS publishes harness runtime images from an AWS-owned account whose ID is not documented. Pinning it would need that ID.
Harness	BedrockMantleInference	arn:aws:bedrock- mantle:us- east-1:118924230273:*	Loose. Every bedrock-mantle resource in the account. Account- and region-scoped, but not resource-scoped. bedrock-mantle is an undocumented internal service, so the valid sub-resource ARNs are unknown. Flagged for follow-up.
Harness	4 × CloudWatch Logs	...log-group:/aws/ bedrock-agentcore/* + the invocation group	Regression — see §0. Should be .../runtimes/* scoped and must not include the invocation log group.
Gateway	GetGateway	...gateway/northstar- assist- gateway-1jkdshz4iy/*	Tight. Scoped to this gateway; the * covers its sub-resources.
Gateway	GetConfigurationBundleVersion...	...configuration-bundle/*	Loose. Any configuration bundle in the account. Read-only, and no bundle is configured for this system, so the statement arguably has no caller and could be removed.
Gateway	GetInferenceProfile	...inference-profile/*	Loose. Read-only metadata on any inference profile in the account. Should be scoped to the embedding model's profile.

Three of these (bedrock-mantle:...:*, configuration-bundle/*, inference-profile/*) are genuinely looser than this system needs and were not narrowed. They are read-only or internal-service actions, so the exposure is low — but “low” is a different claim from “justified,” and §7 now carries them as follow-ups rather than leaving the §2 table’s “0 wildcards” to imply they are not there.

5. Verification — and why it was built in

Over-tightening an agent’s role produces a failure mode worse than leaving it loose: **the agent keeps answering, but without retrieval**. Answers look plausible, are entirely ungrounded, and nothing in the response indicates it.

So --apply is paired with --verify, which after IAM propagation runs one live invocation and checks four things: no error, the guardrail did not block a benign question, the Retrieve tool actually fired, and chunks came back non-zero. **On failure it automatically restores every original policy** — inline and managed — replaying from evidence/iam/before/.

Result:

```
[18:02:14] OK applied ...HarnessExecutionPolicy_le77v [managed]
[18:02:14] OK applied ...GatewayKBAccessProd_7B6CA2 [managed]
[18:02:14] OK applied ...S3PolicyForKnowledgeBase_pcpa3 [managed]
[18:02:14] ==> waiting 15s for IAM propagation before verifying
[18:02:39] OK harness still answers with retrieval after narrowing
```

That single line confirms three things at once: the guardrail is attached and ApplyGuardrail resolves across regions, retrieval still functions through the gateway, and nothing load-bearing was removed. No rollback was needed.

Rollback remains available. All eight original policy documents are preserved, and --restore replays them — including managed policies, via a new default version. The 5-version IAM cap is handled by pruning the oldest non-default version first.

6. What an attacker gains — before versus after

With the original console-generated roles, a principal able to assume the harness execution role could:

- Invoke **any** Bedrock model in the account, including more capable and costlier ones — and any guardrail, knowledge base or inference profile, present or future, via `arn:aws:bedrock:us-east-1:118924230273:*`
- Apply **any** guardrail, including a permissive one, neutralising the control
- Start **browser sessions** — outbound web access under the agent’s identity
- **Execute arbitrary code** via the Code Interpreter
- Mount **EFS and S3 file systems** — persistent read/write storage
- Read and write **agent memory**
- **Enumerate every log group** in the account

And a principal holding the knowledge base role could **cancel foundation-model subscriptions account-wide**, disabling every AI workload in the account.

After narrowing, the same principal can:

- Invoke **one** inference profile and the three regional model ARNs it routes to
- Apply **one** guardrail, via its own cross-Region profile
- Invoke **one** gateway
- Write to **this system’s** log groups
- Publish metrics and traces (no resource-level permissions exist)
- Pull the harness runtime image

The gateway role can read and retrieve from **one** knowledge base and embed with **one** model. The knowledge base role can read **one** bucket, embed with **one** model, and can no longer revoke anything.

Blast radius for a compromise of the agent’s identity goes from “the account’s entire AI estate, plus code execution and outbound network” to “read 30 internal documents.” That is what least privilege actually bought here — not a tidier policy document.

7. Remaining recommendations

1. **Add a `cloudwatch:namespace` condition** to both `PutMetricData` statements, restricting them to `NorthstarAssist/Security`. This is the only surviving wildcard that can be meaningfully constrained.
2. **Add a `permissions` boundary** to all three service roles, so future console operations cannot re-widen them. These roles were generated permissive by default and a console edit could regenerate them that way.
3. **Alert on `PutRolePolicy` / `CreatePolicyVersion` / `AttachRolePolicy`** against these three roles via CloudTrail. Narrowing is only durable if re-widening is visible.
4. **Populate `runtimeUserId`** so invocation logs attribute activity to an employee rather than to the shared execution role — see the monitoring plan §1.5.
5. **Create a break-glass decommissioning role** with credentials independent of the operating ones. During this assessment a credential revocation left live infrastructure running and unreachable; the containment path should not share fate with the operating path.

Added after post-submission review (19 September 2026)

6. **Re-apply the corrected logging statements and re-verify (§0).** This is the first thing to do on any rebuild: restore `runtimes/*` scoping on `CloudWatchLogsGroup`, `CloudWatchLogsStream` and `CloudWatchLogsPutResourcePolicy`, and confirm the harness role holds **no** grant on `/northstar-assist/model-invocations`.
 7. **Make verification check the permissions, not just the behaviour.** `--verify` confirmed the agent still answered with retrieval, which is necessary and insufficient — it cannot detect a widening. The `is_narrower()` guard now catches it at compute time; a post-apply assertion that re-reads each policy and fails on any resource not covered by its original would close the loop.
 8. **Remove `MarketplaceOperationsFromBedrockFor3pModels` entirely** and re-run a knowledge base sync to confirm Titan ingestion is unaffected (§3.3).
 9. **Scope the three loose partial wildcards (§4b):** `inference-profile/*` to the embedding model’s profile, and remove `configuration-bundle/*` and `bedrock-mantle:...:*` if a rebuild confirms they have no caller.
-

Evidence

Artefact	Location
Original policy documents (8)	<code>iam/before/</code>
Narrowed policy documents (9)	<code>iam/after/</code>
Unified diffs, per policy	<code>iam/after/*.diff</code>
Machine-readable change log	<code>evidence/iam/iam_changes.json</code>
Full before/after capture	<code>evidence/iam/iam_before.json</code> , <code>iam_after.json</code>
Generator output	<code>evidence/iam/generated-change-log.md</code>
Narrowing implementation	<code>scripts/20_harden_iam.py</code>